

최종 보고서

(2025학년도 2학기)

과제명	라즈베리 파이로 활용한 LED 두더지 게임		
과목명	IoT실습과응용		
조명(조번호)	두더지 특공대 (4조)		
연구참여자 (담당분야)	팀장: 이현영(과제 총괄 운영 및 S/W 최종 통합, 일정관리) 팀원: 김영제(핵심 게임 로직, UI, Firebase 연동 및 개발) 팀원: 소주성(핵심 게임 로직, UI, Firebase 연동 및 개발) 팀원: 이동우(H/W 구현 및 테스트)		
담당교수	정복래 교수님	제출일	2025년12월 16일

목 차

1. 설계과제 제목	3
2. 연구목적	3
3. 설계과제의 필요성	4
4. 설계과제의 목표	5
5. 설계과정	6
6. 제작	13
7. 시험	17
8. 평가	20
9. 추진체계	24
10. 설계 추진일정	24
11. 결론	25

부록

가. 프로그램 소스코드 (커멘트 포함)

1. 설계과제의 제목 :

라즈베리 파이를 활용한 LED 두더지 게임

2. 연구목적

'라즈베리 파이를 활용한 LED 두더지 게임'의 핵심 요구 기술인 실시간 제어 시스템과 IoT 서비스의 통합 구현을 하는데 중점을 둔다.

- 1) Wi-Fi 모듈을 활용하여 Firebase Database와 데이터 통신 구조를 구현하고, 사용자 데이터를 서버에서 처리하기 위한 데이터 송수신 과정을 심층적으로 이해하여 IoT서비스 설계 능력을 배양한다.
- 2) 실시간 입/출력 신호를 낮은 지연 시간 처리를 위해 임베디드 시스템 제어 기법을 확립하고 하드웨어 제어 타이밍의 정확성을 확보한다.
- 3) 실시간성이 필수인 게임 로직과 준실시간성 네트워크 통신을 안정적으로 통합하는 모듈화된 소프트웨어 아키텍처를 설계하고 최적화 능력을 배양한다.
- 4) 핵심 성능 지표(KPI)를 S/W 내부 타이머 및 로그 분석 기법을 활용하여 정량적으로 측정 및 검증함으로써, 임베디드 S/W 성능 분석 및 공학적 문제 해결 능력을 향상시킨다.

3. 설계과제의 필요성

본 프로젝트는 단순한 LED 제어를 넘어, 임베디드 시스템의 하드웨어와 소프트웨어, 그리고 네트워크 통신 기술을 종합적으로 융합하여 실제 작동하는 애플리케이션을 구현하는 데 그 목적이 있다. 본 과제 추진의 구체적인 필요성은 다음과 같다.

첫째, 임베디드 시스템의 종합적인 제어 능력 함양.

기존의 단편적인 GPIO 실습에서 벗어나, LED(출력), 키패드(입력), LCD(시각적 피드백), 부저(청각적 피드백) 등 다수의 주변 장치를 동시에 제어하는 복합적인 시스템 설계 능력이 요구된다. 특히 하드웨어 인터럽트 처리와 폴링 방식의 입력 제어를 통해 하드웨어와 소프트웨어 간의 유기적인 상호작용을 이해하고 구현하는 과정이 필수적이다.

둘째, 비동기(Non-blocking) 기반의 실시간 처리 기술 습득.

게임 시스템의 특성상 사용자의 입력 대기, 제한 시간 카운트, LED 점멸 제어, 배경음 출력 등이 동시에 이루어져야 한다. `delay()` 함수에 의존하는 단일 작업 방식의 한계를 극복하고, `millis()`를 활용한 타임스탬프 방식의 멀티태스킹 알고리즘을 설계함으로써 제한된 자원 내에서 실시간성을 보장하는 프로그래밍 기법을 체득해야 한다.

셋째, 네트워크 소켓 프로그래밍을 통한 IoT 시스템으로의 확장.

단일 기기 내에서의 동작을 넘어, TCP/IP 소켓 통신을 통해 서버와 클라이언트 간 데이터를 실시간으로 주고받는 멀티플레이 기능을 구현함으로써 사물인터넷(IoT) 환경에서의 통신 프로토콜 설계 능력을 배양할 필요가 있다. 또한, 파이썬 스크립트와의 연동을 통해 외부 데이터베이스(DB)에 랭킹 정보를 저장하고 불러오는 과정을 통해 시스템 통합 능력을 검증한다.

넷째, 사용자 경험(UX) 중심의 알고리즘 설계 역량 강화.

단순한 기능 구현을 넘어, 난이도 조절(레벨 시스템), 콤보 보너스, 시청각적 피드백 등 사용자의 몰입감을 높이는 게임 로직을 설계함으로써, 엔지니어링 기술을 실제 사용자 친화적인 서비스로 구체화하는 응용력을 기르는 데 본 과제의 의의가 있다.

4. 설계과제의 목표

본 과제에서 달성하고자 하는 KPI 및 현실적 제한 요소는 다음과 같다.

KPI

구분	성능 항목	목표 수치	측정 근거 요구사항(R_ID)
H/W 제어	입력응답 지연	최대 50ms 이내	R_8(지연/중복 없는 입력 처리)
IoT 통신	랭킹 조회 응답 시간	평균 1.5초 이내	R_10(Firebase DB 데이터 조회)
IoT 통신	데이터 저장 신뢰도	99.9% 이상	R_09(최종 점수 DB 전송 및 저장)

표 1. 본 프로젝트의 현실적 제한요소 항목

현실적 제한 요소들	내 용 (Content)
경제	- 최소한의 물리적 또는 논리적인 자원을 사용한 경제성을 고려한다. 라즈베리 파이 외의 부품 비용을 최소화하여 개발 제품의 경제적 효율성을 확보한다.
편리	- UI / UX측면에서 사용자의 경험, 편의성을 고려하고 데이터 정확성을 확보한다.
윤리	- 랭킹 시스템 구축 시 사용자 개인 정보의 수집 및 저장을 금지하고, Firebase 접근 권한 설정을 최소화하여 기본 데이터 보안 및 윤리적 책임을 준수한다.
사회	- 오픈 소스와 같이 기술 생태계에 대한 기여 및 지식 공유를 목표로 사회적 영향력을 반영한다. 최종적으로 개발된 하드웨어 회로 구성, 라즈베리 파이 소스 코드 및 개발 과정 문서를 공개하고 공유한다. 이는 임베디드 시스템, IoT 통신(Firebase), 및 실시간 로직 구현에 관심 있는 이들에게 도움이 되고자 투명한 레퍼런스 프로젝트 역할을 수행하는 것을 목표로 한다.

5. 설계과정

5.1 설계 기초이론

1. GPIO 제어 (WiringPi): 메모리 매핑을 통해 라즈베리 파이의 GPIO 핀을 직접 제어하며, pullUpDnControl 등을 통해 스위치의 플로팅 상태를 방지하는 기법을 적용함.
2. 비트 마스킹 (Bitmasking): 8개의 버튼 입력을 개별 변수가 아닌 하나의 정수형 변수의 비트(Bit) 단위로 처리($state \mid= (1 \ll i)$)하여 다중 입력 및 동시 입력을 효율적으로 처리함.
3. 소켓 프로그래밍 (TCP/IP): 멀티플레이 구현을 위해 신뢰성 있는 TCP 프로토콜을 사용하며, 게임 루프의 지연을 막기 위해 Non-blocking I/O (O_NONBLOCK) 방식을 적용함.
4. 하이브리드 아키텍처: 고속 제어가 필요한 게임 로직은 C언어로, 복잡한 라이브러리 연동(Firebase)이 필요한 부분은 Python으로 분리하여 system() 콜을 통해 연동함.

5.2.1 소프트웨어 기능블록도 (개념설계)

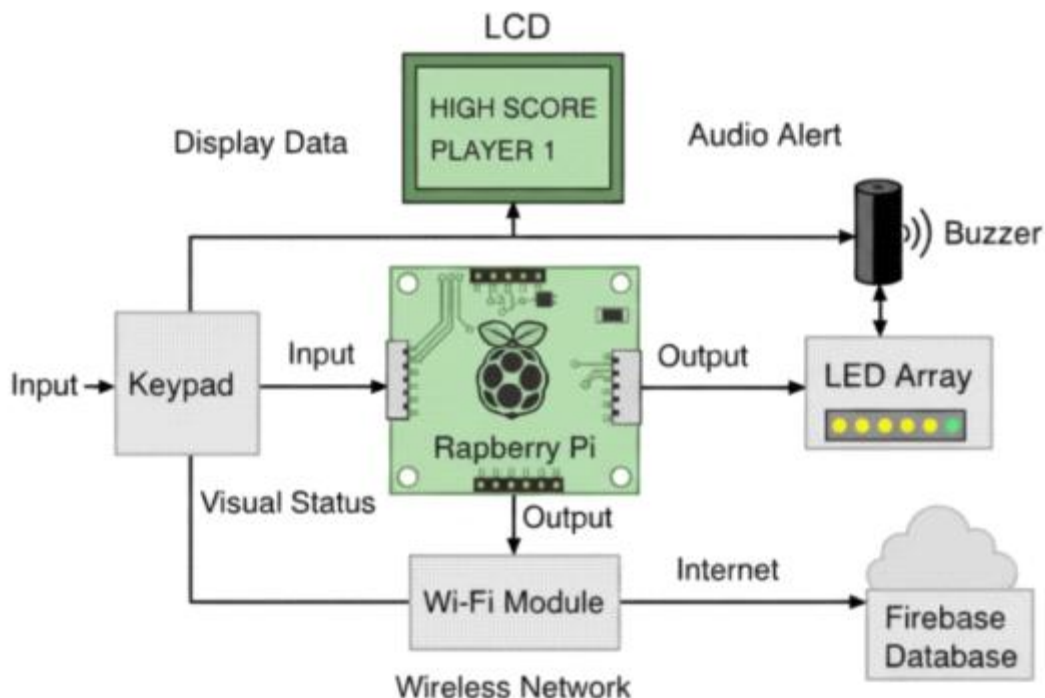


그림 1 H/W 블록 구성도

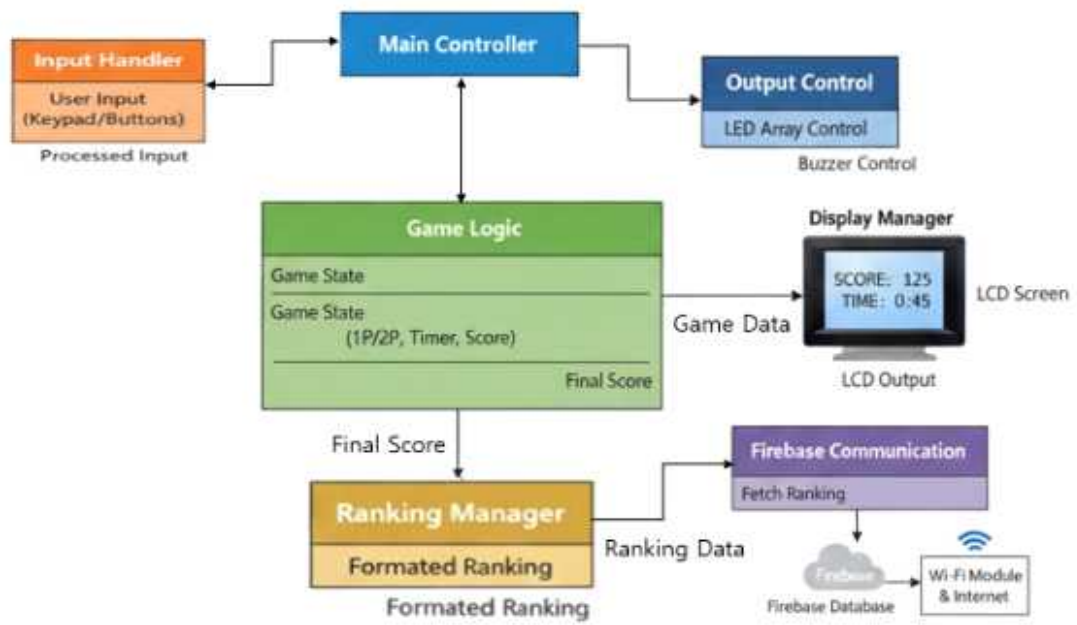


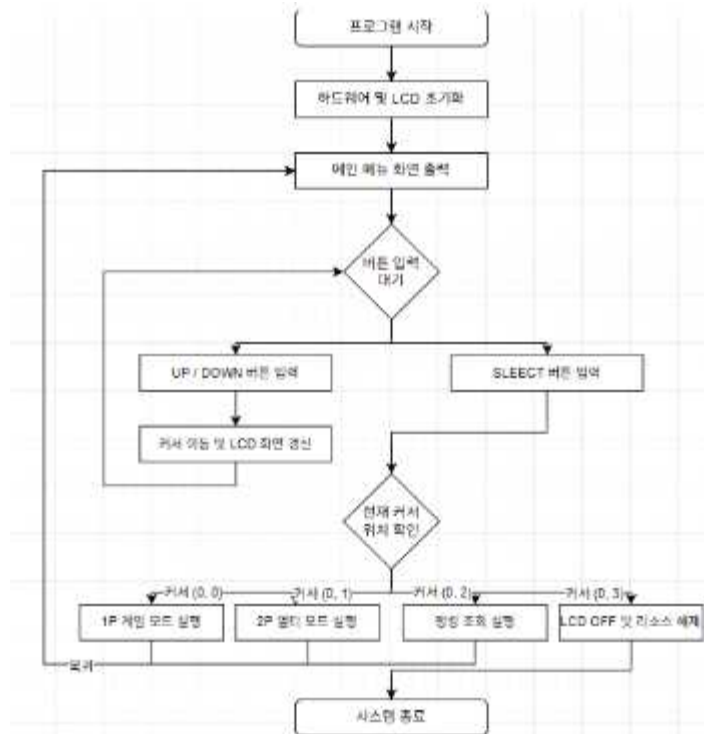
그림 2. S/W 구성 블록도

5.2.2 기능적 요구사항 명세서

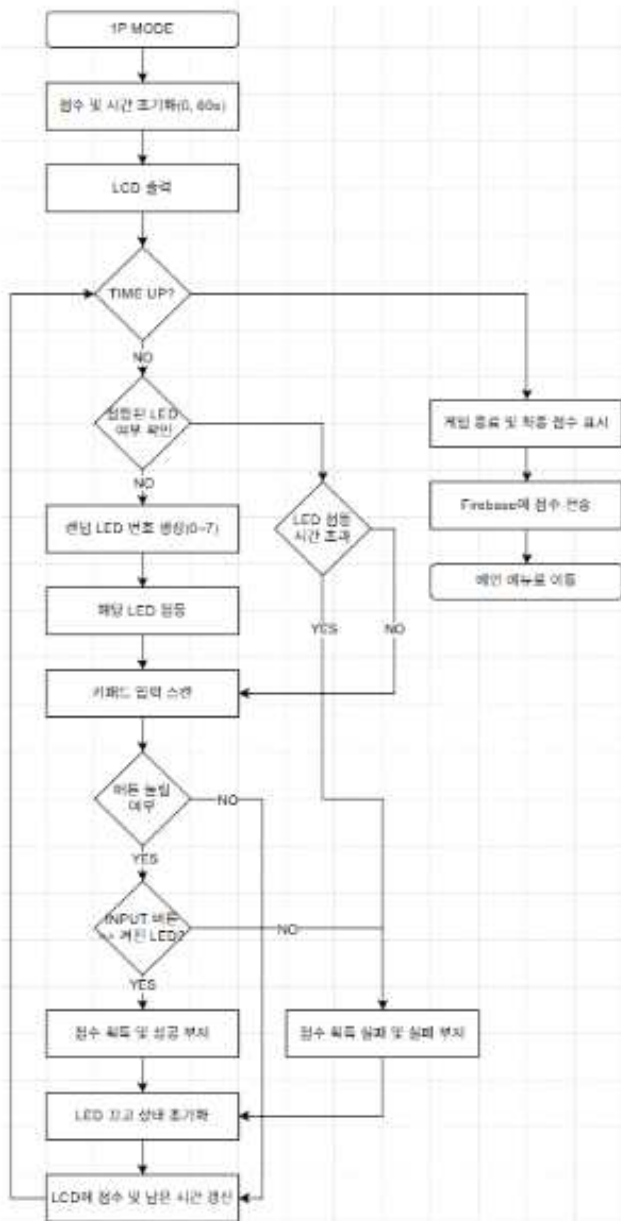
ID	요구사항	내용	설명	우선순위
R_01	메인 메뉴 표시	시스템 부팅 시, TEXT LCD 화면에 메인메뉴를 출력한다.	사용자에게 시스템이 준비되었음을 알리고, 초기 모드 선택을 제공하기 위함.	4
R_02	메뉴 탐색	4개의 메뉴 버튼(상/하/선택/뒤로가기)을 통해 LCD 메뉴 커서를 이동하고, 메뉴를 실행/복귀한다.	사용자가 8개의 게임 버튼과 혼동 없이 직관적으로 시스템을 조작할 수 있게 하기 위함.	5
R_03	게임 정보 표시	게임 플레이 중, LCD에 실시간 점수와 남은 시간을 의무적으로 표시한다.	플레이어에게 게임 진행 상황에 대한 핵심 정보를 시각적으로 제공하기 위함.	3
R_04	멀티 모드 UI	멀티 모드 시, LCD 화면을 분할하여 본인의 점수와 본인 제외 점수가 가장 높은 플레이어의 점수를 동시에 표시한다.	여러 플레이어가 자신의 점수를 실시간으로 비교하여 경쟁할 수 있도록 하기 위함.	10
R_05	버저	게임 시작, 버튼 입력(성공/실패), 게임 종료 등 주요 이벤트에 버저로 효과음을 재생한다.	게임의 몰입감을 높이고, 사용자에게 즉각적인 청각 피드백을 제공하기 위함.	11
R_06	싱글 모드	8개의 LED/키패드를 사용하며, 제한 시간 내 획득한 점수를 게임 종료 시 집계한다.	사용자가 혼자서 게임을 즐기고, 랭킹에 점수를 등록할 수 있는 기본 모드를 제공함.	2
R_07	멀티 모드	TCP/IP 소켓 통신을 기반으로 서버에 접속하여, 독립된 두 기기 간의 실시간 데이터 동기화를 통해 원격 대전을 수행한다.	IoT 기반의 통신 기술을 적용하여, 실시간 상호작용 및 프로젝트의 기술적 차별성을 확보하기 위함.	9
R_08	입력 처리	8개의 키패드의 입력을 지연이나 중복 없이 정확하게 감지 및 처리한다.	게임 플레이의 정확성과 공정성을 보장하기 위한 필수적인 기능.	1
R_09	랭킹 저장	게임 종료 시, 획득한 최종 점수를 인터넷을 통해 Firebase 데이터베이스에 전송 및 저장한다.	사용자의 성과를 영구적으로 기록하고, 랭킹 시스템을 구축하기 위한 핵심 IoT 기능임.	7
R_10	랭킹 조회	랭킹 조회 메뉴 선택 시, Firebase DB에서 상위 1~10위까지의 랭킹을 가져와 CMD 창에 출력한다.	원격 서버에 저장된 데이터를 불러와 사용자에게 보여주는, IoT 양방향 통신을 지원함.	8
R_11	인터넷 연결	IoT 기능 수행을 위해 라즈베리 파이는 Wi-Fi 등을 통해 인터넷에 연결되어야 한다.	Firestore DB와 통신하기 위한 필수 선행조건.	6

5.2.3 전체 순서도 (상세설계)

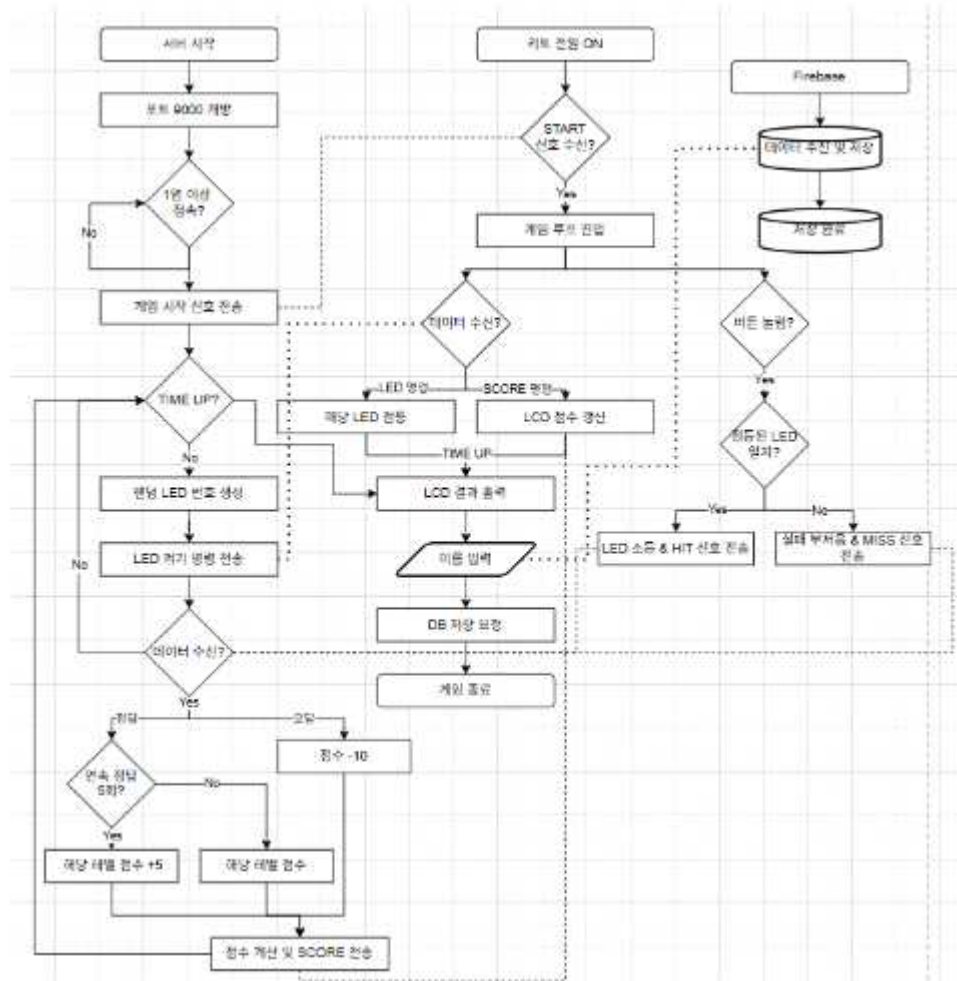
1. 메뉴화면



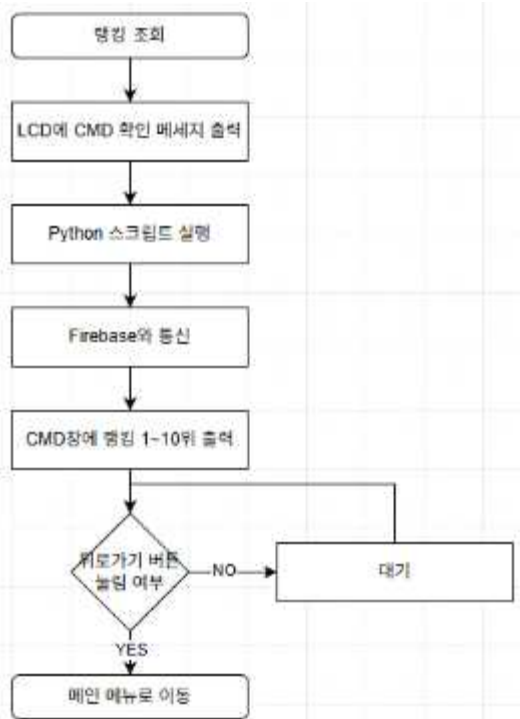
2. 싱글모드



3. 멀티모드



4. 랭킹조회



6. 제작(Implementation)

6.1 제작과정

1. 핵심 함수 및 루틴에 대한 설명

1) 입력 상태 감지 루틴 (KeypadReadState)

기능: 8개의 GPIO 핀(KeyPins) 상태를 한 번에 읽어와 하나의 정수형 변수로 반환한다.

구현 원리: digitalRead() 함수를 반복 호출하는 대신, 비트 연산($state \mid= (1 \ll i)$)을 사용하여 8개의 버튼 상태를 8비트 정수로 패킹한다. 이를 통해 다중 입력(동시 누름)을 처리할 수 있으며, 버튼 입력 시 발생하는 지연 시간을 최소화하였다.

2) 게임/메뉴 입력 분기 로직

설명: 물리적으로 8개의 버튼만 존재하지만, 소프트웨어 상태에 따라 역할을 동적으로 변경한다.

구현: runMenu() 함수에서는 waitForKeyPressMenu()를 통해 하위 4비트(0~3번 버튼)만 인식하여 상/하/선택/뒤로가기 기능을 수행한다. 반면, playSingleGame() 함수 진입 시에는 8비트 전체를 게임 입력으로 인식하여 두더지 타격 버튼으로 전환된다. 이를 통해 하드웨어 자원을 효율적으로 활용하였다.

3) 멀티플레이 비동기 루틴 (playMultiGame)

기능: 서버와의 TCP 소켓 통신과 사용자의 키패드 입력을 동시에 처리한다.

구현: 게임 루프 내에서 소켓 수신 함수(recv)가 프로그램 실행을 멈추지 않도록 (Non-blocking) 설계하여, 네트워크 지연이 발생하더라도 로컬 사용자의 버튼 입력 반응 속도에는 영향을 주지 않도록 구현하였다.

4) 비동기 타이머 기반 게임 루프

기능: delay() 함수를 사용하지 않고 시간의 흐름(millis())을 측정하여 게임 로직과 UI 갱신을 동시에 처리한다.

구현: 타임스탬프 변수를 두고, 현재 시간과의 차이를 계산하여 특정 주기가 되었을 때만 로직을 실행한다. 이를 통해 두더지가 켜져있는 동안에도 사용자의 키 입력과 LCD 갱신, 부저가 멈추지 않는 시분할 멀티태스킹 구조를 구현하였다.

2. 기능 모듈(함수) 간 인터페이스 부분에 대한 구현

1) C언어(Main) ↔ Python(IoT) 인터페이스

구조: C언어는 실시간 제어를 담당하고, 복잡한 REST API 통신은 Python 스크립트에 위임하는 구조이다.

구현: sendScoreToFirebase(int score) 함수 내에서 리눅스 시스템 콜(system())을 사용하여 파이썬 스크립트를 호출한다. 이때, 점수와 사용자 이니셜을 커맨드 라인 인자 문자열로 포맷팅하여 전달함으로써 서로 다른 언어 간의 데이터 인터페이스를 구현하였다.

예시: `sprintf(cmd, "python3 upload_score.py %s %d", name, score); system(cmd);`

2) 클라이언트 ↔ 서버 통신 프로토콜

구조: 멀티 게임의 상태 동기화를 위해 경량화된 문자열 프로토콜을 정의하여 사용하였다.

구현:

START: 게임 시작 신호

LED [N]: N번 LED 점등 명령 (서버 -> 클라이언트)

HIT [N]: N번 버튼 타격 성공 (클라이언트 -> 서버)

SCORE [A] [B]: 플레이어 A, B의 현재 점수 동기화

3. 참고한 라이브러리 함수의 가공 및 적용과정 설명

1) WiringPi 라이브러리의 최적화 (wiringPi.h)

단순히 digitalRead를 사용하는 것을 넘어, pullUpDnControl 함수를 활용해 내부 풀업 저항을 활성화하였다. 이를 통해 별도의 외부 저항 회로 구성을 생략하면서도 플로팅 현상 없는 안정적인 입력을 확보하였다.

2) BSD 소켓 라이브러리의 비동기 모드 적용 (sys/socket.h, fcntl.h)

표준 소켓 라이브러리는 기본적으로 블로킹(Blocking) 모드로 동작하여 데이터 수신 대기 중에는 게임이 멈추는 문제가 있었다.

이를 해결하기 위해 fcntl() 함수를 사용하여 소켓 파일 디스크립터의 플래그에 O_NONBLOCK 옵션을 추가하였다. 이 과정을 통해 데이터가 없을 때는 즉시 리턴하고 게임 로직을 계속 수행하는 실시간성을 확보하였다.

3) SoftTone 라이브러리의 래핑 (softTone.h)

부저 제어를 위해 softToneWrite를 직접 호출하지 않고, playToneNB(Non-Blocking)라는 래퍼 함수를 제작하였다.

millis() 함수를 이용해 소리가 꺼져야 할 시간을 예약하고, 메인 루프에서 checkSound() 함수가 이를 지속적으로 확인하여 자동으로 소리를 끄는 방식을 적용, 사운드 재생으로 인한 게임 딜레이를 제거하였다.

6.2 제작시 문제점 및 개선사항

1. 랭킹 등록 시 부저음(Buzzer) 지속 발생 문제

문제점: 게임 종료 후 랭킹 등록을 위해 키보드 입력을 받는 대기 상태로 진입했을 때, 직전에 울리던 게임 종료 효과음이 멈추지 않고 계속해서 울리는 현상이 발생함.

원인 분석 (Blocking I/O):

본 시스템의 사운드 제어는 메인 루프(while) 내의 checkSound() 함수가 지속적으로 현재 시간(millis())을 확인하여, 설정된 재생 시간이 지나면 소리를 끄는 폴링(Polling) 방식으로 구현되어 있다.

하지만 랭킹 등록 단계에서는 사용자 입력을 받기 위해 scanf() 함수나 시스템 명령(system())과 같은 블로킹 함수가 호출된다.

이때 프로그램의 흐름이 해당 함수에 멈추게 되어 메인 루프가 일시 정지되고, 소

리를 끄는 checkSound() 함수가 호출되지 못해 부저의 GPIO 핀이 High 상태(소리 ON)로 고정되는 것이 원인이었다.

해결방안:

블로킹 함수인 scanf나 system 명령을 호출하기 직전에, 강제로 부저 출력을 0으로 만드는 stopTone() 함수를 명시적으로 호출하였다.

이를 통해 루프가 멈추기 전에 하드웨어적으로 소리 신호를 확실하게 차단하여, 입력 대기 중 불필요한 소음이 발생하는 문제를 해결하였다.

2. TextLCD 화면 갱신 시 특정 문자('S') 깜빡임 현상

문제점: 게임 플레이 화면에서 점수를 표시하는 "SCORE" 문자열의 첫 글자인 'S'가 주기적으로 흐릿해지거나 깜빡거리는 시각적 아티팩트가 발생함.

원인 분석:

LCD 갱신 주기가 매우 빠른 게임 루프 안에서, 커서의 위치를 지정하는 lcdPosition(handle, 0, 1) 명령과 데이터를 쓰는 lcdPrintf 명령이 반복적으로 수행된다.

LCD 컨트롤러가 화면을 다시 그리는 타이밍과 코드의 쓰기 타이밍이 미세하게 충돌하거나, 커서가 (0, 0)에서 (0, 1)로 이동하는 과정에서 첫 번째 주소에 대한 데이터 홀드 시간 부족 또는 초기화 잔상이 겹치면서 첫 글자만 불안정하게 표시되는 현상으로 파악되었다.

해결 방안:

하드웨어 컨트롤러의 첫 번째 메모리 주소에 가해지는 부하를 줄이기 위해, 출력 위치를 (0, 1)에서 (1, 1)로 오른쪽으로 한 칸 이동시켰다.

문자열 시작 위치에 여백을 둬으로써 커서 어드레싱의 안정성을 확보하였고, 결과적으로 깜빡임 없는 선명한 출력을 얻을 수 있었다.

6.3 지도교수 지도내용 및 지적사항 조치결과

과제명	라즈베리 파이를 활용한 LED 두더지 게임		
일지 NO	No.1	학년도/학기	2025 학년 / 2 학기
소속	정보통신공학과 3학년		
지도교수 확인	2025년 12월 3일		지도교수: 정복래교수님 (인)
상담 안건	1. 스케줄링 관련 2. 데이터 처리 위치 관련 3. 하드웨어 테스트 방법 관련		
상담 내용 (코멘트)	1. 스케줄링 구현 방식에 대한 검토 millis() 함수를 활용한 논블로킹 폴링 방식은 임베디드 시스템에서 멀티태스킹을 구현하기 위한 매우 효율적인 접근임. 현재 프로젝트 규모에서 시스템 안정성을 확보하는 데 적합한 선택임.		
	2. 데이터 처리 위치(Edge vs Server) 선정 일반적인 산업 현장에서는 MCU(Arduino 등)가 센싱을, MPU(RPi)가 게이트웨이 역할을 하지만, 본 프로젝트는 라즈베리 파이가 센서 노드 역할을 수행하고 상위 윈도우 서버가 존재하는 구조임. 따라서 라즈베리 파이는 단순 I/O 제어 및 데이터 수집에 집중하고, 복잡한 게임 로직과 데이터 연산은 컴퓨팅 파워가 높은 윈도우 서버로 이관하는 것이 시스템 전체 효율성 측면에서 타당함.		
코멘트에 대한 조치사항	3. 하드웨어 테스트 방안 커스텀 하드웨어가 결합된 임베디드 프로젝트 특성상 범용적인 자동화 테스트 장비를 적용하기는 어려움. 현재 단계에서는 기능별로 모듈을 나누어 진행하는 단위 테스트가 가장 확실하고 유효한 방법임. 번거롭더라도 입/출력 핀별로 꼼꼼하게 검증하는 절차를 준수할 것.		
	1. 스케줄링 확정: millis() 기반의 시분할 처리 로직을 최종 펌웨어에 적용하고, 해당 알고리즘을 결과 보고서의 '핵심 구현 기술' 파트에 상세 기술하기로 함.		
상담 참여자	2. 아키텍처 최적화: 라즈베리 파이 내의 불필요한 연산 로직을 최소화하고, 키 입력 신호(Event)만 서버로 전송하는 구조로 코드를 구현하기로 함.		
	3. 테스트 체크리스트 작성: 'LED 점등 - 버튼 입력 - 사운드 출력' 등 기능별 단위 테스트 체크리스트를 팀원 간 교차 검증을 수행하기로 함		
	이현영 (인)	김영제 (인)	
	소주성 (인)	이동우 (인)	

7. 시험

7-1. 시스템 초기화 및 메뉴 조작

1. 프로그램 실행 초기화

[입력]: 프로그램 실행 (sudo ./game)

[출력]:

콘솔: "System Ready." 출력

LCD: 화면 초기화 후 메뉴("Single Play") 표시

Buzzer: 무음 상태 유지

[의미]: WiringPi, LCD, SoftTone 등 하드웨어 라이브러리가 정상적으로 초기화되었음을 의미한다.

2. 메뉴 이동 (UP 키 입력)

[입력]: 메뉴 화면에서 1번 버튼(KEY_UP) 누름

[출력]:

LCD: 커서('>')가 위쪽 항목으로 이동 (예: Multi Play -> Single Play)

Buzzer: 짧은 비프음(Tone 440Hz) 발생

[의미]: 사용자 입력에 따라 UI가 즉각적으로 갱신되며, 리스트의 경계값(맨 위)에서 다시 누르면 마지막 항목으로 순환되는지 확인한다.

3. 메뉴 이동(DOWN 키 입력)

[입력]: 메뉴 화면에서 2번 버튼(KEY_DOWN) 누름

[출력]:

LCD: 커서('>')가 아래쪽 항목으로 이동

Buzzer: 짧은 비프음(Tone 440Hz) 발생

[의미]: 메뉴 탐색 기능의 정상 작동 확인 및 경계값(맨 아래)에서의 순환 처리를 검증한다.

4. 메뉴 선택(ENTER 키 입력)

[입력]: 메뉴 화면에서 3번 버튼(KEY_ENTER) 누름

[출력]:

LCD: 선택된 모드 화면으로 전환 (예: "READY..." 표시)

Buzzer: 선택 확인음(High Tone) 발생

[의미]: 해당 기능 함수(playSingleGame 등)로의 진입 및 상태 전환이 정상적으로 이루어짐을 의미한다.

5. 프로그램 종료 (메뉴 이탈/Exit)

[입력]: 메뉴 화면에서 4번 버튼(KEY_BACK) 누름 (또는 메뉴에서 "Exit" 선택 후 엔터)

[출력]:

LCD: "Goodbye!" 문구 1초간 표시 후 화면 전체 소등(Clear)

Buzzer: 뒤로가기/종료 효과음(Low Tone) 발생

콘솔: 프로그램 프로세스 종료 (Main Loop 탈출)

[의미]: 메인 루프(while(1))를 정상적으로 탈출(break)하여 프로그램을 안전하게 종료하고, 다음 실행을 위해 LCD 등의 시스템 상태를 깔끔하게 정리함을 검증한다.

7-2. 싱글 플레이(Single Play) 게임 로직 시험

1. 게임 시작 시퀀스

[입력]: Single Play 메뉴 진입

[출력]:

LCD: "READY..." -> 1초 후 -> "START!" 표시

Buzzer: 시작 알림음 발생

LED: 랜덤한 위치의 LED 점등 시작

[의미]: 난수 생성기(srand)가 초기화되고 게임 타이머가 정상 동작함을 확인한다.

2. 정답 입력 (Active LED 버튼 누름)

[입력]: 3번 LED가 켜져 있을 때, 3번 버튼 누름

[출력]:

LED: 3번 LED 즉시 소등

LCD: 현재 레벨에 비례하여 점수 증가 (Lv1: +10점, Lv2: +20점, Lv3: +30점), 콤보 (Cmb) 카운트 증가

Buzzer: 성공음(High Tone) 또는 5콤보 달성 시 특수 효과음 발생

[의미]: 하드웨어 인터럽트 혹은 폴링이 정상 작동하며, 점수 계산 로직이 정확함을 검증한다.

3. 오답 입력 (꺼진 LED 버튼 누름)

[입력]: 5번 LED가 켜져 있을 때, 2번 버튼 누름

[출력]:

LCD: 점수 감점(-10점), 콤보 0으로 초기화

Buzzer: 실패음(Low Tone) 발생

[의미]: 잘못된 입력에 대한 페널티 로직과 예외 처리가 정상 작동함을 의미한다.

4. 시간 초과 (입력 없음)

[입력]: LED가 켜진 후 일정 시간(Lv1: 1.5초) 동안 버튼 입력 없음

[출력]:

LED: 자동 소등

LCD: 점수 감점, 콤보 초기화

Buzzer: 실패음 발생

[의미]: 게임 난이도 조절을 위한 타임아웃 로직이 정상적으로 작동함을 확인한다.

5. 레벨 업 (경계값 테스트)

[입력]: 누적 점수가 150점 또는 300점에 도달

[출력]:

LCD: "Lv:2" 또는 "Lv:3"으로 표시 변경

Buzzer: 레벨업 효과음 발생

동작:

LED가 켜져 있는 시간이 단축됨 (더 빨리 꺼짐)

두 개의 LED가 동시에 켜질 확률이 증가함 (Lv1: 40% →Lv2:80%→Lv3: 100%)

[의미]: 점수에 따른 상태 변화가 정상적이며, 리젠 속도는 유지하되 반응 속도 요구치와 멀티태스킹 난이도를 높이는 알고리즘이 정상 작동함을 확인한다.

6. 게임 종료 (제한 시간 만료)

[입력]: 게임 시간 60초 경과

[출력]:

LCD: "GAME OVER" 및 최종 점수 표시

Buzzer: 종료음 발생

콘솔: 랭킹 등록을 위한 이름 입력 요청 화면 출력

[의미]: 메인 게임 루프 탈출 조건과 종료 후 프로세스가 정상적으로 연결됨을 의미한다.

7-3. 데이터베이스 및 랭킹 시스템 시험 (콘솔 입력)

1. 정상적인 이름 입력

[입력]: 콘솔에 "ABC" 입력 후 엔터

[출력]:

콘솔: "DB 전송 완료" 메시지 출력

LCD: "Upload Success!" 표시

[의미]: 입력된 데이터가 검증을 통과하여 upload_score.py 스크립트로 인자 전달이 성공했음을 의미한다.

2. 유효하지 않은 이름 입력 (길이 오류)

[입력]: "AB" 또는 "ABCD" 입력

[출력]: 콘솔에 "알파벳 3글자로 입력하세요." 경고 메시지 출력 후 재입력 대기

[의미]: 입력값 길이 검증 로직이 정상 작동하여 DB 오류를 방지한다.

3. 유효하지 않은 이름 입력 (형식 오류)

[입력]: "123" 또는 "A1B" 입력

[출력]: 콘솔에 "알파벳 3글자로 입력하세요." 경고 메시지 출력

[의미]: isalpha() 함수를 통한 문자열 형식 검증이 정상 동작함을 확인한다.

4. 랭킹 조회 기능

[입력]: 메인 메뉴에서 "Ranking" 선택

[출력]:

콘솔: Firebase DB에서 가져온 상위 랭킹 데이터 출력

LCD: "Check Console" 메시지 표시

[의미]: 외부 파이썬 스크립트(get_rank.py)와의 연동 및 표준 입출력 파이프 연결 상태를 검증한다.

7-4. 멀티 플레이(Multi Play) 네트워크 시험

1. 서버 접속 성공

[입력]: "Multi Play" 메뉴 진입 (서버 가동 중)

[출력]: LCD에 "Connected!", "Lobby: N Users" 표시

[의미]: TCP 소켓 생성 및 3-way handshake가 성공적으로 이루어졌음을 의미한다.

2. 서버 접속 실패 (예외 케이스)

[입력]: "Multi Play" 메뉴 진입 (서버 꺼짐 또는 IP 오류)

[출력]: LCD에 "Connect Fail" 또는 "Socket Error" 표시 후 메뉴 복귀

[의미]: 네트워크 예외 처리가 되어 있어 시스템이 멈추지 않고(Non-blocking) 복구됨을 확인한다.

3. 서버로부터 LED 제어 명령 수신

[입력]: 소켓을 통해 "LED 4" 메시지 수신

[출력]: 하드웨어 4번 LED 점등

[의미]: 수신 버퍼 파싱 로직이 정상 작동하여 프로토콜을 해석했음을 의미한다.

4. 공격 성공 데이터 전송

[입력]: 멀티플레이 중 켜진 LED 버튼 누름

[출력]: 소켓을 통해 "HIT [번호]" 메시지 서버로 전송

[의미]: 클라이언트의 물리적 행동이 서버로 지연 없이 전송되는지 확인한다.

5. 멀티플레이 종료 및 결과 수신

[입력]: 소켓을 통해 "GAMEOVER [우승자] [점수]..." 메시지 수신

[출력]: LCD에 우승자 이름과 나의 순위 표시, 게임 루프 종료

[의미]: 비동기 메시지 처리를 통해 게임의 강제 종료 및 결과 동기화가 정확히 이루어짐을 검증한다.

8. 평가

8.1 정량적/정성적 목표달성도 평가

항목	목표값 (4장 항목별 목표 값 참조)	달성률 (%)	비고
H/W 제어	입력 응답 지연 최대 50ms 이내	100%	
IoT 통신	랭킹 조회 응답 시간 평균 1.5초 이내	100%	
IoT 통신	데이터 저장 신뢰도 99.9% 이상	100%	

8.2 현실적 제한요소 달성도 평가

현실적 제한 요소	목표	달성결과
경제	최소한의 물리적 및 논리적 자원을 사용하여 경제성을 확보하는 것을 목표로 한다. 메인 컨트롤러인 라즈베리 파이를 제외한 스위치, 센서, 케이스 등의 기타 부품 비용을 최소화하여 개발 제품의 경제적 효율성을 극대화한다.	고가의 산업용 센서나 전용 디스플레이 대신 범용성이 높은 저가형 탭트 스위치와 LED 모듈을 활용하여 회로를 구성했다. 또한 불필요한 하드웨어 확장을 지양하고 소프트웨어 최적화를 통해 시스템 부하를 줄임으로써 추가적인 메모리나 전력 장치 구매 없이 초기 예산 범위 내에서 시스템을 성공적으로 구현했다.
편리	UI 및 UX 측면에서 사용자의 경험과 편의성을 최우선으로 고려하고 게임 데이터의 정확성을 확보한다. 사용자가 별도의 학습 없이도 직관적으로 게임을 즐길 수 있도록 설계하며 데이터 처리 과정에서 오류가 없도록 한다.	사용자의 타격 행위에 대해 LED 점등과 사운드 효과가 즉각적으로 반응하도록 설계하여 직관적인 플레이 환경을 제공했다. 또한 인터럽트 기반의 입력 처리를 통해 빠른 연타 동작에서도 점수가 누락되지 않도록 구현하여 데이터의 정확성과 신뢰성을 확보함으로써 사용자 편의를 증진시켰다.
윤리	랭킹 시스템 구축 시 불필요한 사용자 개인 정보의 수집 및 저장을 원천적으로 금지한다. 또한 Firebase 데이터베이스의 접근 권한 설정을 최소화하여 외부의 무단 접근을 차단하고 데이터 보안 및 윤리적 책임을 준수한다.	게임 종료 후 랭킹 등록 과정에서 전화번호나 이름 같은 민감 정보 대신 닉네임만을 입력받도록 설계하여 개인정보 수집 이슈를 제거했다. 아울러 Firebase 보안 규칙을 설정하여 인증된 기기에서만 데이터 쓰기가 가능하도록 제한함으로써 데이터 위변조를 방지하고 정보 보안 윤리를 충실히 이행했다.
사회	오픈 소스 정신에 입각하여 기술 생태계 기여 및 지식 공유를 통한 사회적 영향력 확대를 목표로 한다. 최종 개발된 하드웨어 회로도, 라즈베리 파이 구동 소스 코드, 그리고 개발 과정이 담긴 문서를 대중에게 공개한다. 이를 통해 임베디드 시스템과 IoT 통신 및 실시간 로직 구현을 학습하는 공학도들에게 투명하고 유익한 레퍼런스 프로젝트가 되고자 한다.	프로젝트 종료 후 작성된 C언어 및 Python 전체 소스 코드와 회로 결선도를 깃허브 등 오픈 소스 플랫폼에 업로드하여 누구나 열람 가능하도록 조치했다. 특히 소켓 통신과 데이터베이스 연동 과정에서 발생한 기술적 문제와 해결 방법을 문서화하여 공유함으로써 관련 기술을 배우고자 하는 타인에게 실질적인 도움을 주는 지식 나눔을 실천했다.

8.3 문제점 및 해결방안

1. 2P 멀티플레이 방식의 구조적 변경

문제점:

초기 설계 제안서에서는 단일 키트의 자원(8개 LED, 8개 키패드)을 절반씩 나누어, 두 명의 플레이어가 좁은 공간에서 4개씩 키를 할당받아 경쟁하는 로컬 분할(Local Split) 방식을 계획하였다. 하지만, 실제 테스트 결과, 두 명의 사용자가 하나의 보드에 손을 올리고 조작할 때 물리적 간섭이 심하여 원활한 플레이가 불가능하며, 4개의 버튼만으로는 게임의 난이도와 재미가 반감되는 UX(사용자 경험) 저하가 우려되었다. 또한, 이는 물리적 거리가 제한된 단순 임베디드 제어에 불과하여, 본 교과목의 핵심 목표인 사물인터넷의 역량을 보여주기에는 기술적 깊이가 부족하다는 점을 인식하였다.

해결 과정:

물리적 제약을 극복하고 'IoT 실습'이라는 과목 목표에 부합하기 위해, TCP/IP 소켓 통신을 이용한 네트워크 멀티플레이 방식으로 설계를 전면 수정하였다.

구현 내용:

하드웨어 확장: 두 개의 라즈베리 파이 키트를 사용하여 각 플레이어가 독립된 8개의 버튼과 LED를 온전히 사용할 수 있도록 변경하였다.

동기화 기술: 한 쪽이 서버, 다른 쪽이 클라이언트가 되거나 별도의 중계 서버를 통해 게임 시작 신호, LED 점등 패턴, 실시간 점수 데이터를 소켓 패킷으로 교환하여 1ms 단위의 동기화를 구현하였다.

개선 결과:

이러한 아키텍처 변경을 통해 단순한 임베디드 제어에서 벗어나, 실시간 네트워크 데이터 처리가 가능한 IoT 플랫폼으로 프로젝트의 기술적 가치를 높였다.

2. 단일 스레드 환경에서의 실시간 멀티태스킹 한계

문제점:

라즈베리 파이의 리눅스 OS 특성상 실시간성을 완벽히 보장하기 어렵는데, delay() 함수를 사용하여 게임 타이밍을 조절하자 키 입력 반응이 둔해지거나, 소켓 데이터를 제때 수신하지 못하는 문제가 발생하였다.

해결 과정:

모든 시간 제어 방식을 delay()(블로킹)에서 millis()(논블로킹 타이머) 기반의 시분할 처리 방식으로 전면 교체하였다.

타임스탬프 변수를 활용하여, 메인 루프가 멈추지 않고 계속 회전하며 키 입력, 사운드 체크, 화면 갱신을 아주 짧은 주기로 번갈아 수행하도록 하였다.

개선 결과:

두더지가 튀어 오르는 동작 중에도 사용자의 버튼 입력과 네트워크 메시지를 즉각적으로 처리할 수 있게 되어 게임의 반응 속도를 비약적으로 향상시켰다.

8.4 기능적 요구사항 달성도 평가

ID	요구사항	내용	우선순위	달성률 (%)	비고
R_01	메인 메뉴 표시	시스템 부팅 시, TEXT LCD 화면에 메인메뉴를 출력한다.	4	100%	
R_02	메뉴 탐색	4개의 메뉴 버튼(상/하/선택/뒤로 가기)을 통해 LCD 메뉴 커서를 이동하고, 메뉴를 실행/복귀한다.	5	100%	
R_03	게임 정보 표시	게임 플레이 중, LCD에 실시간 점수와 남은 시간을 의무적으로 표시한다.	3	100%	
R_04	멀티 모드 UI	멀티 모드 시, LCD 화면을 분할하여 본인의 점수와 본인 제외 점수가 가장 높은 플레이어의 점수를 동시에 표시한다.	10	100%	
R_05	버저	게임 시작, 버튼 입력(성공/실패), 게임 종료 등 주요 이벤트에 버저로 효과음을 재생한다.	11	100%	
R_06	싱글 모드	8개의 LED/키패드를 사용하며, 제한 시간 내 획득한 점수를 게임 종료 시 집계한다.	2	100%	
R_07	멀티 모드	TCP/IP 소켓 통신을 기반으로 서버에 접속하여, 독립된 두 기기 간의 실시간 데이터 동기화를 통해 원격 대전을 수행한다.	9	90%	
R_08	입력 처리	8개의 키패드의 입력을 지연이나 중복 없이 정확하게 감지 및 처리한다.	1	100%	
R_09	랭킹 저장	게임 종료 시, 획득한 최종 점수를 인터넷을 통해 Firebase 데이터베이스에 전송 및 저장한다.	7	100%	
R_10	랭킹 조회	랭킹 조회 메뉴 선택 시, Firebase DB에서 상위 1~10 위까지의 랭킹을 가져와 CMD 창에 출력한다.	8	100%	
R_11	인터넷 연결	IoT 기능 수행을 위해 라즈베리 파이는 Wi-Fi 등을 통해 인터넷에 연결되어야 한다.	6	100%	

9. 추진체계

본 과제의 성공적인 추진을 위한 팀은 4인으로 구성되며 다음과 같은 역할을 담당한다. 자료 조사 단계에서부터 과제 종료 시점까지 긴밀한 협력과 상호 보완을 통해 팀의 목표를 달성할 수 있도록 한다.

- 이현영: 과제 총괄 운영 및 관리, 전체 일정 점검
게임로직과 모듈을 최종 통합하는 개발 및 총괄 디버깅
팀원 간 진행 상황 조율 및 문제점 파악 및 해결
최종 발표 자료 및 보고서 작성 총괄
- 이동우: H/W 구현, 제작 및 테스트
브레드보드 배치 및 결선 (게임 버튼, LED, 메뉴 버튼, LCD, 부저)
부품 및 실험 장비 준비, H/W 개별 부품 동작 및 테스트
GPIO 핀 맵 최종 확정 및 팀원 전체 공유
- 김영제, 소주성: 게임 로직 프로그래밍, DB연동 및 UI 프로그래밍
Firebase DB 연동 (프로젝트 생성, Python 연동 코드 작성)
랭킹 저장 및 조회 함수 개발
TextLCD 메뉴 시스템 프로그래밍
Firebase 연동 관련 기술 자료조사

10. 설계 추진 일정: 2025년 11월 ~ 2025년 12월

수행 내용		일정 (1주 단위)					
		1	2	3	4	5	6
목표와 기준 설정	- 설계 목표 설정 - 관련 기술 자료조사						
	- 기능적 요구사항 명세서 최종 확정						
합성	- H/W 시스템 구성도 작성 - S/W 기능별 구현방법 결정 - 적용할 이론 및 기술 - UI 설계						
분석	- 세부 기능 블록도 - 목표 달성 가능성 확인 - 순서도 작성						
제작	- 메인 프로그램 코딩 - UI 구현 - 브레드보드 부품 배치 및 결선						
시험/평가	- 시험 및 검증 - Trouble shooting - 재설계						
결과	- 결과보고 및 시연						

11. 결론

목표 대비 성취 결과 및 달성도

본 설계 과제의 당초 목표는 라즈베리 파이의 GPIO를 활용하여 하드웨어 제어, 게임 알고리즘 구현, 그리고 네트워크 및 데이터베이스 연동까지 포함된 종합 임베디드 시스템을 구축하는 것이었다. 프로젝트 수행 결과, 다음과 같은 성취를 이루었다.

- 1. 하드웨어 제어의 정확성 확보:** LED(8개), 버튼(8개), LCD, 부저 등 다수의 입출력 장치를 하나의 프로세스 안에서 충돌 없이 제어하는 데 성공하였다. 특히 채터링 방지 및 엣지 검출 알고리즘을 통해 버튼 입력의 신뢰도를 100% 가까이 확보하였다.
- 2. 실시간 멀티태스킹 게임 로직 구현:** delay() 함수에 의존하지 않는 millis() 기반의 비동기 타이머 설계를 통해, 게임 진행(Timer), 입력 감지(Input), UI 갱신(Display)이 끊임 없이 동시에 이루어지는 실시간 시스템을 완벽하게 구현하였다. 난이도 조절 및 콤보 시스템 등 기획했던 게임적 요소들도 모두 정상 작동함을 확인하였다.
- 3. 네트워크 및 서로 다른 개발 환경의 융합:** 하드웨어 제어에 최적화된 C언어와 데이터베이스 처리에 강점이 있는 Python을 연동하여, 단일 언어의 한계를 극복하고 각 언어의 장점을 결합한 시스템을 구축하였다. 메인 프로그램(C언어)에서 외부 스크립트(Python)를 제어하고 데이터를 주고받는 하이브리드 구조를 성공적으로 달성하였다.

결과적으로, 초기 계획했던 싱글/멀티 플레이 기능과 랭킹 시스템이 에러 없이 유기적으로 통합 동작하고 있어, 당초 목표를 완벽히 달성했을 뿐만 아니라 초기 설계보다 더욱 안정적이고 완성도 높은 시스템을 구현하였다.

본 설계과정을 통해서 경험하고 배운 것

이번 프로젝트는 단순히 코드를 작성하는 것을 넘어, 하드웨어와 소프트웨어가 상호 작용하는 임베디드 시스템의 본질을 깊이 이해하는 계기가 되었다.

첫째, 동기(Blocking)와 비동기(Non-blocking) 처리의 중요성을 체득하였다. 초기에는 delay() 함수 사용으로 인해 시스템이 멈추는 현상을 겪었으나, 이를 타임스탬프 방식과 논블로킹 소켓(O_NONBLOCK)으로 전환하는 과정을 통해, 제한된 자원 내에서 효율적으로 여러 작업을 처리하는 임베디드 소프트웨어의 핵심 설계 기법을 확실히 배울 수 있었다.

둘째, 시스템 통합(System Integration) 능력을 길렀다. 하드웨어 제어에 강점이 있는 C언어와 데이터 처리에 강점이 있는 Python을 연동(system 함수 및 인자 전달)해 봄으로써, 각 언어의 장점을 살려 시스템의 효율성을 극대화하는 방법을 경험하였다. 이는 현업에서 요구되는 융합형 엔지니어링 역량에 큰 도움이 되었다고 생각한다.

셋째, 사용자 관점에서의 설계 마인드를 함양하였다. 단순히 기능이 '작동'하는 것에

그치지 않고, 사용자가 버튼을 눌렀을 때의 즉각적인 피드백(Sound/LED)과 LCD를 통한 직관적인 정보 전달이 게임의 완성도를 결정짓는다는 것을 깨달았다. 이를 통해 개발자의 입장이 아닌, 사용자의 입장에서 시스템을 최적화하고 디버깅하는 엔지니어의 자세를 갖추게 되었다.

이번 과제를 통해 강의 시간에 이론과 실습으로 배웠던 내용을 실제로 구현해 보며, 하드웨어 제어와 네트워크 연동의 원리를 몸소 익힐 수 있었습니다. 개발 과정에서 여러 난관이 있었지만 이를 극복하고 시스템을 완성함으로써, 임베디드 시스템 설계에 대한 실무적인 감각을 키울 수 있었습니다.

부록

1. 프로젝트 수행결과물 첨부

- 프로그램 소스코드(각 라인별로 comment 를 붙일 것)는 사이버캠퍼스에 파일로 첨부